第1回　ローカルLLMを使って、AIの中をのぞく

ChatGPT などのAIが、文章をどう作り、どう「考えて」いるのかを見ていきます。クラウドのAIでは見えない「答えを作る途中」も、自分のPCで動くローカルLLMなら手元でのぞけます。講師の画面に合わせて、同じコマンドを自分のPCで実行してください。

時間	使うもの	教材
60分	**事前準備で入れたもの**	**zipをダウンロード**

Webページのバックアップ（2026-11-09時点）。折りたたみ部分（実行結果の例・解説・もっと試す・困ったとき）は、すべて開いて載せています。

0 なぜローカルLLMを使うのか 5分

ChatGPT や Claude は、提供している会社のサーバーで動いています。ローカルLLMは、同じ仕組みのAIを自分のPCで動かすものです。



Ollama　モデルを動かすソフト。事前準備で入れたもので、裏で常に動き、チャットやプログラムからの質問を受け付けます。

モデル　AIの本体のファイル（数GB）。今日使う `qwen3.5:0.8b` と `qwen3.5:4b`（メモリ8GBの人は `qwen3.5:2b`）のことです。

	クラウドのAI	ローカルLLM
どこで動くか	提供している会社のサーバー	自分のPC
賢さ	大きくて賢いモデルが使える	PCのメモリで動く大きさに限られ、賢さは落ちる
データ	入力したデータは社外に出る	ネットを切っても動き、データは外に出ない
中が見えるか	中で何が起きているかは、ほとんど見えない	**確率や思考の途中など、中身を直接のぞける**

この勉強会でローカルLLMを使うのは、最後の点のためです。クラウドのサービスでは見えにくい「AIが文章を作る途中」を、手元で何度でも、料金を気にせず試せます。

1 動作確認 10分

事前準備が終わっている人は、教材のフォルダ（`llm-agent-mcp-handson`）でターミナルを開き、次の2つを実行してください。フォルダの開き方は、事前準備ページの手順5と同じです。終わっていない人は、モデルのダウンロードに時間がかかるため、この時間内には追いつけません。今日は講師の画面を見ながら進め、勉強会のあとで事前準備ページの手順を進めてください。

ターミナル（教材のフォルダで）

```
> ollama list
> uv sync
```

ollama list の表示の例（列や順番は PC によって違います）

```
NAME            ID      SIZE    MODIFIED
qwen3.5:4b      …
qwen3.5:0.8b    …
```

表示の細かいところは、PCやOSによって違います。確かめるのは次の2点だけです。

- `ollama list`：`qwen3.5:0.8b` と `qwen3.5:4b`（メモリ8GBの人は `qwen3.5:2b`）の2つが出ている
- `uv sync`：`error` と出ずに終わる（出るメッセージは、初めてか2回目かなどで変わります）

このあとのスクリプトは、入っている方のモデル（4b か 2b）を自動で使います。

2 プログラムから呼ぶ 10分

チャット画面で質問するとき、裏ではプログラムがデータを送り、返事を受け取っています。その中身をそのまま表示してみます。

予想 **「『おはようございます』を英語にして。訳だけ答えて」と送ると、返事はどんな形で届くでしょうか。答えがまとめて1回で届くでしょうか。**

ターミナル

```
> uv run session1/2.1_api.py
```

▼ 実行結果の例を見る

```
── 送ったデータ ──────────────────────
{
  "model": "qwen3.5:4b",
  "messages": [
    {
      "role": "user",
      "content": "「おはようございます」を英語にして。訳だけ答えて"
    }
  ],
  "stream": true,
  "think": false
}

── 返ってきたデータ（届いた順）──────
{"done":false,"message":{"role":"assistant","content":"Good"}}
{"done":false,"message":{"role":"assistant","content":" morning"}}
{"done":false,"message":{"role":"assistant","content":"."}}
{"done":true,"done_reason":"stop","total_duration":2394087875,"load_duration":2
117282042,"prompt_eval_count":26,"prompt_eval_duration":97976750,"eval_count":3
,"eval_duration":153954667,"message":{"role":"assistant","content":""}}

── まとめると ────────────────────────
Good morning.

書いたトークン数: 3  かかった時間: 0.15秒（1秒に約19トークン）
```

観察 **表示を見て、次を確かめてください。**

- 送ったデータの `messages` には、何が入っているか
- 返事は何行に分かれて届いたか。1行に何文字ずつか
- 最後の行（`"done":true`）には、何の数字が入っているか

解説を開く

チャットは「会話の履歴を送って、続きを書いてもらう」だけ。 `messages` は、誰が（`role`）何を言ったか（`content`）の一覧です。2回目の質問をするときは、1回目の質問と答えも含めて、履歴をまるごと送り直します。LLMは前の会話を覚えているわけではありません（「もっと試す」の2.3で確かめられます）。

返事は数文字ずつ届く。 LLMは答えを一度に作っているのではなく、少しずつ書き足しています。この1回分の単位を「トークン」と呼びます。ChatGPT の画面で文字が少しずつ出てくるのは、この仕組みのためです。

最後の行は「明細」。 `prompt_eval_count` は送ったトークンの数、`eval_count` は書いたトークンの数、`eval_duration` は書くのにかかった時間（ナノ秒）です。クラウドのAIの料金も、この「送った数」と「書いた数」の両方で決まります。

短い質問でも、送ったトークンは26ある。 質問の前後に「ここから user の発言」「ここで発言が終わり」といった目印が足され、会話の形に整えてから送られるためです。

待ち時間のほとんどは、モデルの読み込み。 `load_duration` は、モデルのファイルをメモリに読み込む時間です。例では、全体（`total_duration`）の約2.4秒のうち約2.1秒が読み込みで、書いている時間は0.15秒だけでした。読み込んだモデルはしばらくメモリに残るので、続けて実行すると速くなります。

もっと試す

好きな質問で試す

2.1のスクリプトに、好きな質問を渡せます。返事の長さによって、届く行の数がどう変わるか見てみてください。

ターミナル

```
> uv run session1/2.1_api.py "好きな質問"
```

> わかること　返事が長いほど、届く行が増えます。どんなに長い答えでも、数文字（1トークン）ずつ書き足している点は同じです。

2.2 OpenAI互換の書き方と見比べる

Ollamaは、ChatGPTのAPIと同じ形でも呼べます。`2.1_api.py` と送るデータ・返ってくるデータの形を比べてみてください。接続先とモデル名（とAPIキー）を変えれば、同じコードがChatGPTなどでも動きます。

ターミナル

```
> uv run session1/2.2_openai_compat.py
```

▼ 実行結果の例を見る

```
── 送ったデータ ──────────────────
{
  "model": "qwen3.5:4b",
  "messages": [ …2.1と同じ… ],
  "reasoning_effort": "high"
}

── 返ってきたデータ ──────────────
{
  "choices": [
    {
      "finish_reason": "stop",
      "message": {
        "content": "Good morning",
        "role": "assistant",
        "reasoning": "Thinking Process:\n\n1.  **Analyze the Request:** …"
      }
    }
  ],
  "model": "qwen3.5:4b",
  "usage": {
    "completion_tokens": 397,
    "prompt_tokens": 24,
    …
  }
}
```

わかること　包み方が違うだけで、中身は2.1と同じです。答えは `choices` の中の `content`、書いたトークン数は `usage` に入ります。この例は思考ありなので、2語の答えのために397トークン考えています。

2.3 会話の履歴を確かめる

1回目に「好きな飲み物はほうじ茶」と伝え、2回目に「好きな飲み物は何でしたか？」と聞きます。2回目を、1回目のやりとりを付けて送った場合と、付けずに送った場合で、答えを比べてください。画面には、それぞれ送った `messages` も表示されます。

ターミナル

```
> uv run session1/2.3_history.py
```

わかること　履歴を付けると「ほうじ茶」と答えられます。付けないと、「覚えていない」と答えるか、当てずっぽうの答え（検証では「コーヒー」）を返します。LLMが会話を「覚えている」ように見えるのは、プログラムが毎回履歴を送り直しているからです。会話が長くなるほど、送るトークンも増えます。次回のAgentも、この履歴にツールの結果を書き足していく仕組みです。

3 次の単語の予測 15分

LLMは、1トークンずつ書き足しています。では、次に何を書くかはどう決めているのでしょうか。

3.1 次の1つを見る

予想　**「日本の首都は」の次に来るのは何でしょう。それは何%くらいの確率でしょう。**

ターミナル

```
> uv run session1/3.1_next_token.py
```

▼ 実行結果の例を見る

```
モデル: qwen3.5:4b  入力: 日本の首都は
東京          65.9%
、             6.5%
「             4.8%
なぜ           4.0%
？             1.5%
日本           1.0%
どこ           0.7%
東京都         0.6%
京都           0.5%
どれ           0.5%
上位10個の合計: 86.0%（残りは、ほかの何万もの候補）
実際に選ばれたのは: 、
```

3.2 答え全体を、1トークンずつ見る

予想　**「新しいカフェの名前を1つ考えて」と頼むと、LLMはどこで迷うでしょうか。**

ターミナル

```
> uv run session1/3.2_answer_tokens.py
```

▼ 実行結果の例を見る

```
モデル: qwen3.5:4b  答え: カクチャイタマ

#    選ばれた          確率  ほかの候補
1    カ               0.9%  カフェ 9%  ミ 6%  ア 3% ← 迷った
2    ク               2.2%  プチ 39%  フ 9%  ペ 9% ← 迷った
3    チャ             0.6%  テル 51%  テ 6%  ティ 3% ← 迷った
4    イ              15.2%  （書き終わり） 34%  [改行] 7%  ウ 5% ← 迷った
5    タ               0.5%  （書き終わり） 49%  [改行] 7%  フ 4% ← 迷った
6    マ               3.9%  （書き終わり） 47%  [改行] 8%  チ 7% ← 迷った

6トークンのうち、迷った所（確率50%未満）： 6か所
```

観察

- 確率がほぼ100%のトークンと、迷ったトークン（50%未満）は、それぞれどこか
- 「(書き終わり)」が候補に出てきたのはどこか
- ほかの人と比べて、同じ答えになったか

迷わない質問とも比べてみてください。次の質問では、「東京」が98%前後になります。

ターミナル

```
> uv run session1/3.2_answer_tokens.py "日本の首都を一言で答えて"
```

「(文字の一部)」は、珍しい漢字を書かせると出やすくなります。どの文字が、いくつのトークンに分かれたか見てください。

ターミナル

```
> uv run session1/3.2_answer_tokens.py "「円すい」をすべて漢字で書いて。漢字だけ答えて"
```

解説を開く

LLMがしているのは「次のトークンの予測」だけ。これまでの文章をもとに、次に来るトークンの候補それぞれに確率を付け、その中から1つ選んで書き足します。これを、答えが終わるまで繰り返しています。

確率が一番高いものが、必ず選ばれるわけではない。確率に応じてくじ引きのように選ぶので、同じ質問でも答えが変わります（くじ引きの度合いは、temperature という設定で変えられます。「もっと試す」で試せます）。迷った所（確率が割れた所）で別の候補が選ばれると、その先の文章も変わっていきます。

トークンは文字とは限らない。「東京」「 morning」のようによく出る言葉は1トークンにまとまり、珍しい文字は2つに分かれることがあります。「(文字の一部)」は、1文字の前半か後半だけのトークンです。

書き終えるタイミングも予測している。「(書き終わり)」は、ここで答えを終える、という特別なトークンです。LLMは「次の文字」と同じように「ここで終わるかどうか」も確率で選んでいます。カフェの名前の例では、4トークン目以降、毎回3〜5割の確率で終わろうとしていました。

もっと試す

ほかの書きかけの文で試す

3.1のスクリプトに、好きな書きかけの文を渡せます。もう一度実行すると、選ばれるものが変わることもあります。

ターミナル

```
> uv run session1/3.1_next_token.py "むかしむかし、あるところに"
```

▼ 実行結果の例を見る

```
モデル: qwen3.5:4b  入力: むかしむかし、あるところに

、          ████████████████████████████████████████  81.2%
森          ▌                                          1.2%
村          ▌                                          1.2%
、「         ▍                                          0.7%
[改行]      ▍                                          0.6%
猫          ▍                                          0.6%
「          ▍                                          0.4%
宮          ▍                                          0.4%
川          ▏                                          0.3%
国王        ▏                                          0.3%

上位10個の合計: 86.7%（残りは、ほかの何万もの候補）
実際に選ばれたのは: 、
```

わかること　決まり文句の続きは、1つの候補に確率が集まります（「、」が81%）。一方、そのあとに来そうな言葉（森・村・猫…）は、どれも1%前後に割れています。

同じ質問を何度も

3.2のスクリプトを3回実行します。毎回同じ所で迷うでしょうか。答えは変わるでしょうか。

ターミナル

```
> uv run session1/3.2_answer_tokens.py
```

▼ 実行結果の例を見る

```
1回目  答え: トピコ
2回目  答え: エモート
3回目  答え: ミラージュ・カフェ

1トークン目の候補（3回とも同じ）
  カフェ 9%  ミ 6%  ア 3% …
実際に選ばれたのは
  ト（1.0%） → エ（0.8%） → ミ（6.5%）
```

> わかること　答えは毎回変わります。1トークン目の候補と確率は3回とも同じで、違うのはくじ引きで選ばれたものだけです。最初に1%ほどの候補が選ばれると、その先の名前もまるごと変わります。

くじ引きをやめる

3.2のスクリプトに `--temperature 0` を付けると、くじ引きをしなくなります。何度か実行して、答えが変わるか確かめてください。

ターミナル

```
> uv run session1/3.2_answer_tokens.py --temperature 0
```

> わかること　temperature（温度）は、くじ引きの度合いを決める設定です。0にすると毎回同じ答えになり、大きくするほど確率の低い候補も選ばれやすくなります。ChatGPTなどのAPIにも同じ設定があります。なお、画面の確率はモデルが出した元の確率で、実際に選ぶときはほかの設定による調整も加わります。そのため、くじ引きなしでも、いちばん上の候補が選ばれるとは限りません。

3.3 画像を読ませる

qwen3.5 は画像も読めます。画像もいったんトークンに変換され、文章と同じように「次のトークンの予測」で説明が書かれます。付属の画像のほか、自分の画像のパスを渡しても試せます。

ターミナル

```
> uv run session1/3.3_image.py
```

▼ 実行結果の例を見る

```
モデル: qwen3.5:4b
画像  : sample.png

これは「ローカルLLMの動き方を学ぶ勉強会」のポスターで、ollamaコマンドによる会話の実例とモデル比較（0.8bと4b）が含まれています。
日程は11/xx 19:00〜と記載されており、画面にはコーヒー豆も描かれています。
```

> わかること　画像の中の文字（日程・モデル名・コマンド）は、かなり読めています。一方で、マグカップを「コーヒー豆」と書くような見間違いや、「3行で」と頼んだのに2行で返すこともあります。付属の画像（`session1/assets/sample.png`）を開いて見比べてみてください。

3.4 PDFを読ませる

Ollamaには、PDFなどのファイルをそのまま渡す機能はありません。ChatGPTなどの「ファイル添付」も、アプリが裏でPDFから文字（または画像）を取り出し、それをトークンにしてからLLMに渡しています。付属のPDF（架空の社内図書館の利用案内）から文字を取り出して渡し、要約させてみてください。画面には、実際にLLMに渡した文字も表示されます。

ターミナル

```
> uv run session1/3.4_pdf.py
```

▼ 実行結果の例を見る

```
── LLM に渡した文字（PDF から取り出したもの）──
社内図書館 利用案内
2026 年度版　総務部 図書館係（架空の資料・勉強会用）
開館時間と場所
場所 本社 3 階 西側ラウンジ横
開館時間 平日 9:00〜19:00（祝日・年末年始は休館）
…
新刊コーナー
毎月第 1 営業日に新しい本を入れます。リクエストは図書館係までどうぞ。毎月 5 冊まで受け付けます。

── 返事 ──────────────────────────
開館は平日 9:00〜19:00、貸出は 1 回だけ延長可能ですが期限超過は利用不可です。
本に書き込みや付箋の貼付をせず、汚損・欠損時は図書館係へ連絡してください。
新刊コーナーへのリクエストは毎月第 1 営業日に受け付けられ、2 週間以内で返却可能です。
```

> **わかること** LLMに渡るのは、取り出した文字だけです（表は1行ずつの文字になっています）。要約は大筋で合っていますが、3行目は「新しい本を入れる日」「リクエスト」「貸出期間」が混ざった誤りです。ファイルを添付して要約させたときも、元の文書と見比べる必要があります。

4 小さいモデルと大きいモデルの考え方を比べる 15分

最近のLLMの多くは、答える前に「考えている途中の文章」（思考）を書きます。同じ質問を2つのモデルに投げ、思考と答えを比べます。1問で1〜1分半ほどかかります（小さいモデル約25秒、大きいモデル30秒〜1分。Macで測った目安で、PCによってはもっとかかります）。

用意した質問

1. 英単語 strawberry の中に、r は何個ありますか？
2. 池の睡蓮は毎日2倍に増え、48日目に池全体を覆いました。池の半分を覆ったのは何日目ですか？
3. 本とブックカバーを合わせて1100円です。本はブックカバーより1000円高いです。ブックカバーは何円ですか？

スクリプトに番号を付けて実行すると、その質問を2つのモデルに投げます。

予想 **質問1を投げると、2つのモデルはそれぞれ正しく答えられるでしょうか。思考が長いのはどちらでしょうか。**

ターミナル

```
> uv run session1/4.1_compare.py 1

▼ 最後に出る比較の例を見る

■ qwen3.5:0.8b
［答え］
（答えを書く前に打ち切りました）

■ qwen3.5:4b
［答え］
英単語 strawberry には、r が 3 個あります。

■ くらべると
  qwen3.5:0.8b    思考  6718文字  書いたトークン  2048  時間  26.8秒（打ち切り）
  qwen3.5:4b      思考  2127文字  書いたトークン   858  時間  32.7秒
```

観察 **上にスクロールして、それぞれの［考えていること］を読んでください。思考は英語で書かれています。全部読む必要はなく、流れをつかめば十分です。**

- 小さいモデルは、なぜ答えにたどり着けなかったのか
- 大きいモデルは、途中で自分の考えを確かめたり、言い直したりしているか（"Wait," を探してみてください）

解説を開く

思考も「次のトークンの予測」で書かれた文章。思考は特別な頭脳の働きではなく、答えの前に書く下書きです。ただ、下書きを書くと、そのあとのトークンはその下書きも踏まえて予測されます。途中の計算や確認を文章として書き出すことで、正しい答えにたどり着きやすくなります。

大きいモデルほど、下書きの質が上がる。検証したときの思考の一部です。小さいモデルは綴りを間違えたまま確認を繰り返し、上限まで書いても答えにたどり着きませんでした。大きいモデルは、一度数えたあとに綴りを確かめ直して答えています。

qwen3.5:0.8b 2048トークンで打ち切り

```
Wait, is that right? "strawberry".
s-t-r-a-w-b-e-r-y.
Yes. So r appears twice. Wait.
…
Wait, no. "strawberry".
Wait, 'b' comes after 'r'?
s-t-r-a-w-b-e-r-y. Yes. …
```

qwen3.5:4b 858トークンで回答

```
* r - (yes, 1st 'r')
…
* r - (yes, 2nd 'r')
* r - (yes, 3rd 'r')
Total count = 3.
…
Wait, let me double-check the spelling.
s-t-r-a-w-b-e-r-r-y. Yes, three 'r's.
```

思考が英語になるのは、はっきりした理由は公開されていませんが、学習に使われた文章の多くが英語であることが一因と考えられています。答えは質問に合わせて日本語で書きます。

その分、時間がかかる。思考も書いたトークンとして数えられます。簡単な質問なら、小さいモデルや思考なしで十分なこともあります。

もっと試す

質問2・3でも比べる

用意した質問の2と3でも、同じように違いが出るか確かめます。

ターミナル

```
> uv run session1/4.1_compare.py 2
> uv run session1/4.1_compare.py 3
```

▼ 最後に出る比較の例を見る

```
質問2（睡蓮）
■ qwen3.5:0.8b ［答え］（答えを書く前に打ち切りました）
■ qwen3.5:4b   ［答え］池の半分を覆ったのは47日目です。
■ くらべると
  qwen3.5:0.8b   思考  7446文字  書いたトークン  2048  時間  25.5秒（打ち切り）
  qwen3.5:4b     思考  2366文字  書いたトークン   885  時間  31.8秒

質問3（ブックカバー）
■ qwen3.5:0.8b ［答え］（答えを書く前に打ち切りました）
■ qwen3.5:4b   ［答え］ブックカバーの価格は 50 円 です。
■ くらべると
  qwen3.5:0.8b   思考  5899文字  書いたトークン  2048  時間  25.5秒（打ち切り）
  qwen3.5:4b     思考  3057文字  書いたトークン  1338  時間  47.9秒
```

わかること　3問とも同じ傾向です。小さいモデルは上限の2048トークンまで考えても答えにたどり着かず、大きいモデルは正解しました。例では、質問2の大きいモデルの思考に「48÷2＝24日目」というよくある間違いへの言及もありました。

思考なしで答えさせる

同じモデルで、思考ありと思考なしの答えを比べます。思考がないと、答えはどう変わるでしょうか。

ターミナル

```
> uv run session1/4.1_compare.py 1 --no-think
```

▼ 実行結果の例を見る

```
■ qwen3.5:0.8b
［答え］
英語の「strawberry」の日本語訳をみると、1 つですね。
…

■ qwen3.5:4b
［答え］
単語「strawberry」を見ると、4 個の r が含まれています。
…

■ くらべると
  qwen3.5:0.8b    思考      0文字  書いたトークン    136  時間    1.7秒
  qwen3.5:4b      思考      0文字  書いたトークン    161  時間    5.9秒
```

> わかること　思考なしだと数秒で返ってきますが、例ではどちらのモデルも間違えました（1個、4個）。この質問では、答えの前に下書き（思考）を書くことが正解に効いています。結果は実行のたびに変わるので、何度か試してみてください。

自分の質問で比べる

仕事で使いそうな質問を2つのモデルに投げ、「小さいモデルで足りる質問」と「大きいモデルが要る質問」を1つずつ探してみてください。業務データは入れず、言い換えた架空の内容で試します。

ターミナル

```
> uv run session1/4.1_compare.py "自分の質問"
```

> わかること　言い換えや短い要約のような質問は、小さいモデルでも足りることが多いです。数える・計算する・条件を組み合わせるような質問ほど、大きいモデルと思考の差が出ます。

5 まとめ 5分

- **チャットは、会話の履歴を送って、続きを書いてもらう仕組み**
- **LLMは、次のトークンを確率で選んで書き足すことを繰り返している**
- **「考える」も、答えの前に下書きを書くこと。大きいモデルほど、下書きで自分を正せる**

各手順の「もっと試す」は、時間が余ったときや勉強会のあとに。答えの提出は不要です。気づいたことは、全回終了後のアンケートで聞きます。

次回はAgent。LLMが「このツールを使いたい」と書き、プログラムがそれを実行して結果を返す、という往復を作ります。ツールを使うかどうかも、LLMにとっては「次に何を書くか」の1つです。

勉強会のあとにPCのメモリを空ける方法は、事前準備ページの最後の「勉強会が終わったら」にあります。

うまくいかないとき

当日の手順の途中で困ったときの対処です。

「Ollama に接続できません」と出る

Ollamaのアプリが起動していません。Windowsはスタートメニュー、Macはアプリケーションフォルダから Ollama を起動してから、もう一度実行してください。

「見つかりません」とモデル名が出る

モデルのダウンロードが終わっていません。`ollama pull qwen3.5:0.8b` などを実行してください（事前準備ページの手順2）。

ファイルやライブラリが見つからないと英語で出る

`can't open file` や `No `pyproject.toml` found` と出るときは、教材のフォルダ以外でターミナルを開いています。`session1` フォルダと `pyproject.toml` が入っているフォルダで開き直してください。

Windows で ZIP を展開すると、`llm-agent-mcp-handson` フォルダの中に、同じ名前のフォルダがもう1つできることがあります。その場合は、中の方のフォルダで開いてください。

`No module named 'ollama'` と出るときは、`python` で実行しています。`uv run session1/…` の形で実行してください。

「確率が返ってきませんでした」と出る

Ollama が古いため、確率を返せません。ollama.com/download から最新版を入れ直してください。上書きで大丈夫です（ダウンロードしたモデルは消えません）。

チャット（ollama run）で考え込んで、答えが返ってこない

普通のチャットには、思考の上限がありません。特に小さいモデルは、同じ確認を繰り返して何分も考え続けることがあります。Ctrl＋C で止めてください。

思考なしで試したいときは、`ollama run qwen3.5:4b --think=false` のように起動します（メモリ8GBの人は `qwen3.5:2b`）。チャット中なら `/set nothink` と入力しても切り替えられます。

教材のスクリプトは、思考なし（2・3）か、思考の上限つき（4）で動くので、このようにはなりません。

4.1 がなかなか終わらない

大きいモデルは、思考が長いと1〜2分かかることがあります。同じことを繰り返して止まらないときは、小さいモデルは2048トークン、大きいモデルは4096トークンで自動的に打ち切ります。途中でやめたいときは Ctrl＋C を押してください。

`uv` が見つからないと言われる

ターミナルを閉じて開き直してください。それでもだめなら、事前準備ページの手順4をやり直してください。